

ECE 2011

Introduction to Computer Engineering
Spring 2026

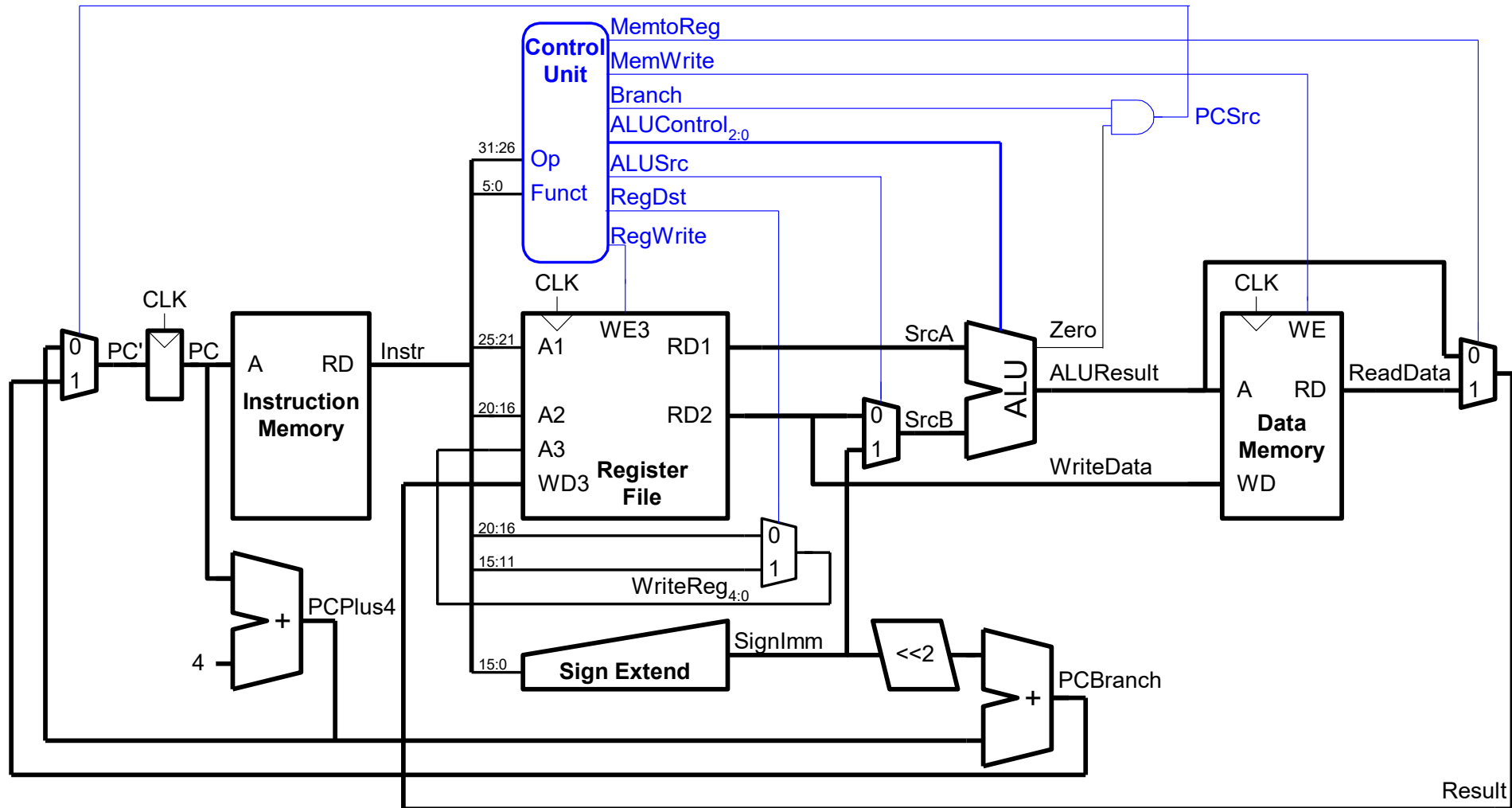
Lecture 19

04-07-2026

Linghao Song

Yale ECE

Single-Cycle Processor



Processor Performance

- Program execution time

Execution Time = (#instructions)(cycles/instruction)(seconds/cycle)

- Definitions:

- CPI: Cycles/instruction

- clock period: seconds/cycle

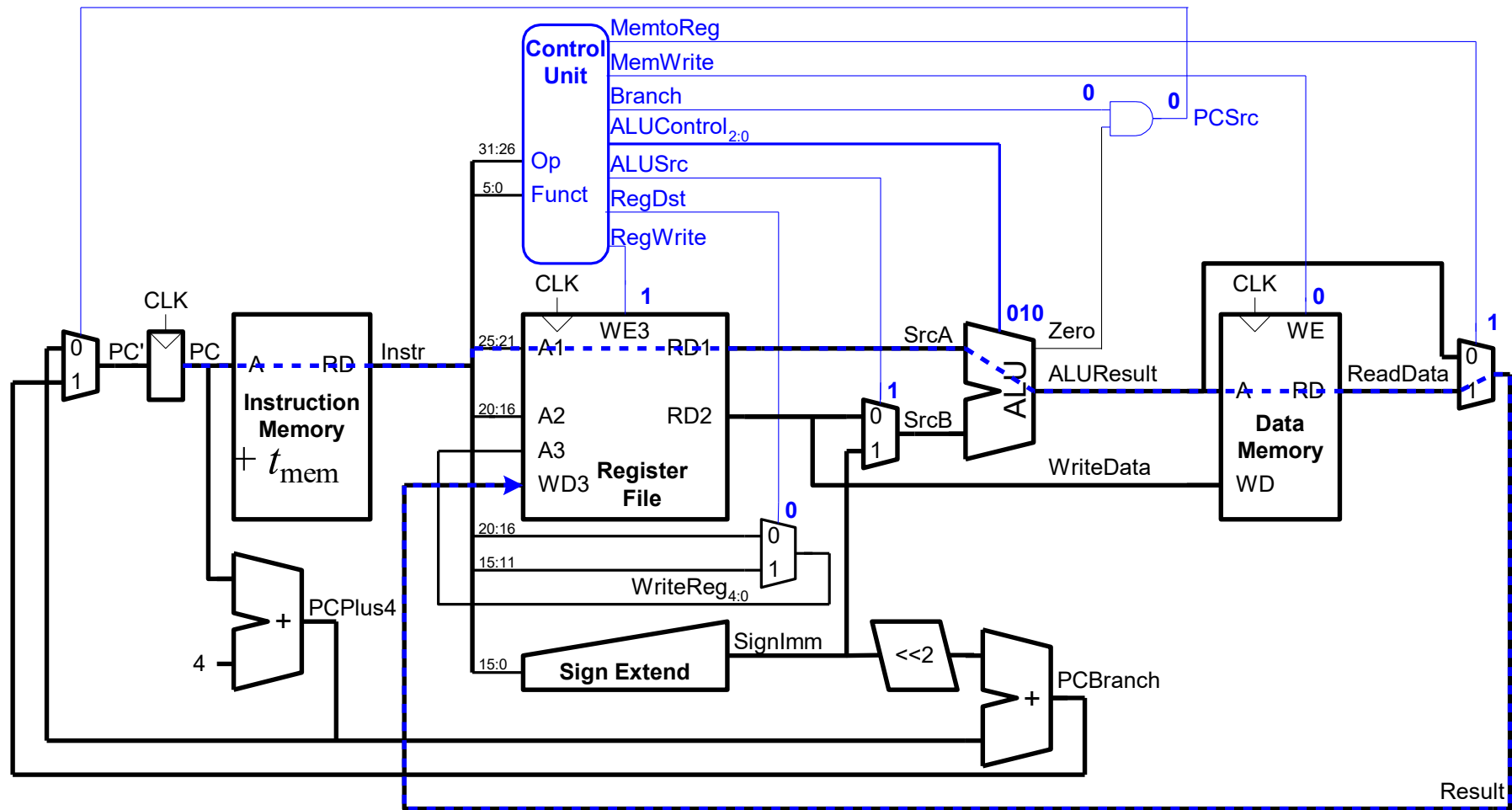
- IPC: instructions/cycle = IPC

$$\text{execution time} = \frac{\#instr}{(instr/second)}$$

$$\frac{instr}{second} = \frac{instr/cycle}{second/cycle} = \frac{IPC}{clock\ period.}$$

- Challenge is to satisfy constraints of:
 - Cost
 - Power
 - Performance

Single-Cycle Performance



T_C limited by critical path (1w)

Single-Cycle Performance

- Single-cycle critical path:

$$T_c = t_{pcq_PC} + t_{mem} + \max(t_{RFread}, t_{sext} + t_{mux}) + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

- Typically, limiting paths are:

- memory, ALU, register file

- $T_c = t_{pcq_PC} + t_{mem} + t_{RFread} + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$

$$= t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup}$$

Single-Cycle Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$T_c = ?$$

$$= t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup}$$

$$= [30 + 2(250) + 150 + 25 + 200 + 20] \text{ ps}$$

$$= 925 \text{ ps}$$

Single-Cycle Performance Example

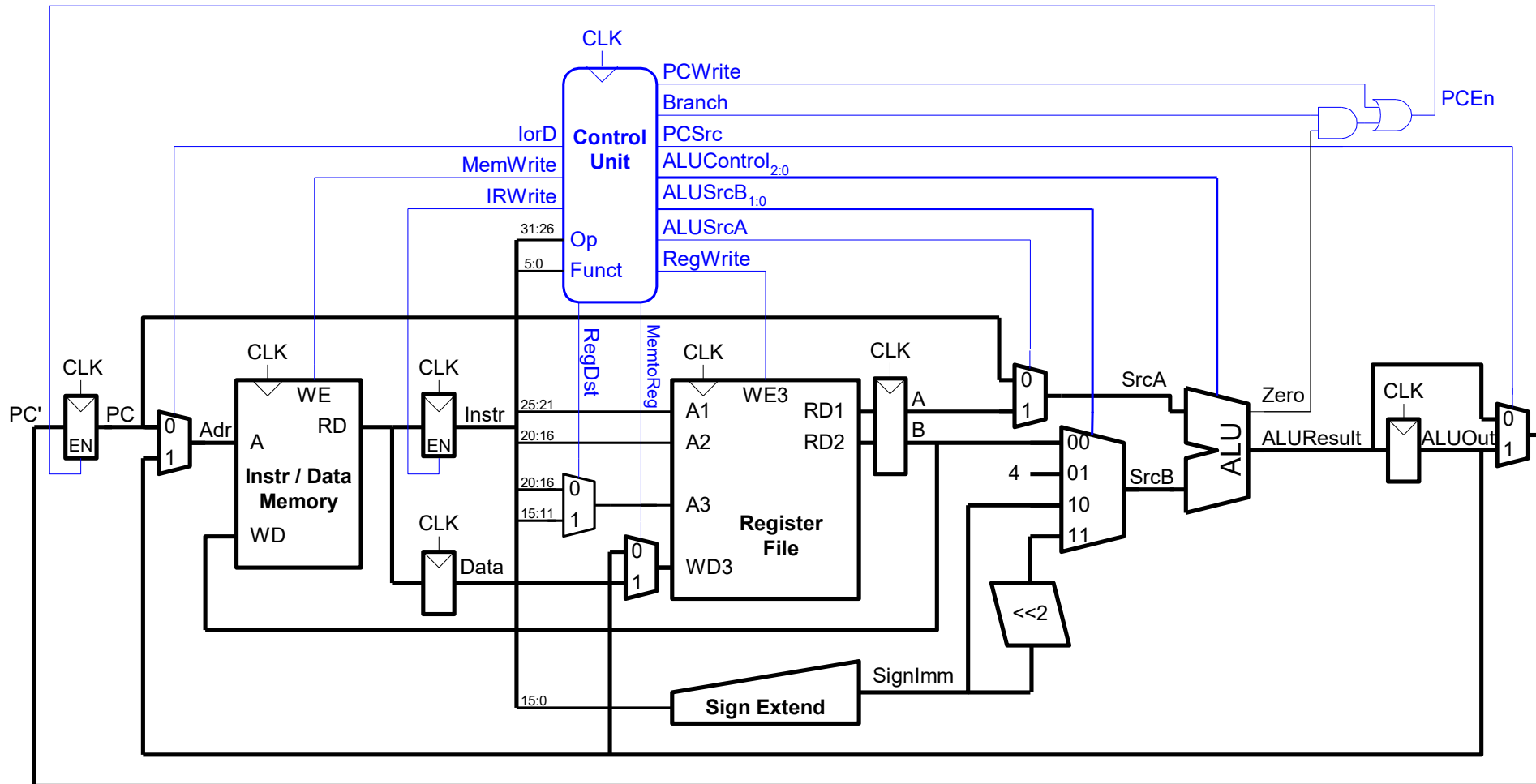
Program with 100 billion instructions:

$$\begin{aligned}\text{Execution Time} &= \# \text{ instructions} \times \text{CPI} \times T_C \\ &= (100 \times 10^9)(1)(925 \times 10^{-12} \text{ s}) \\ &= \mathbf{92.5 \text{ seconds}}\end{aligned}$$

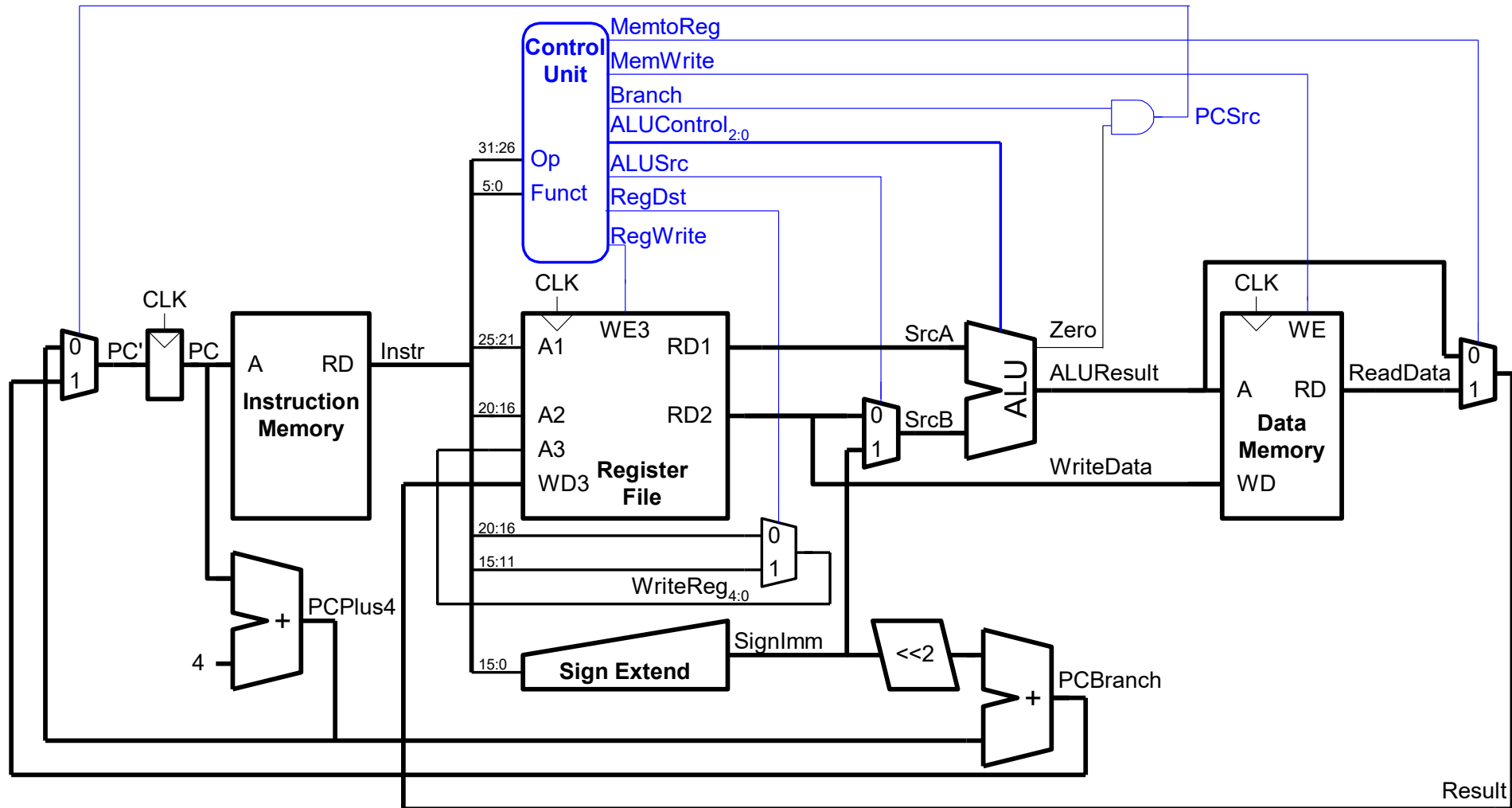
Multicycle MIPS Processor

- **Single-cycle:**
 - + simple
 - cycle time limited by longest instruction (1_w)
 - 2 adders/ALUs & 2 memories
- **Multicycle:**
 - + higher clock speed
 - + simpler instructions run faster
 - + reuse expensive hardware on multiple cycles
 - sequencing overhead paid many times

Multicycle Processor



Single-Cycle Processor

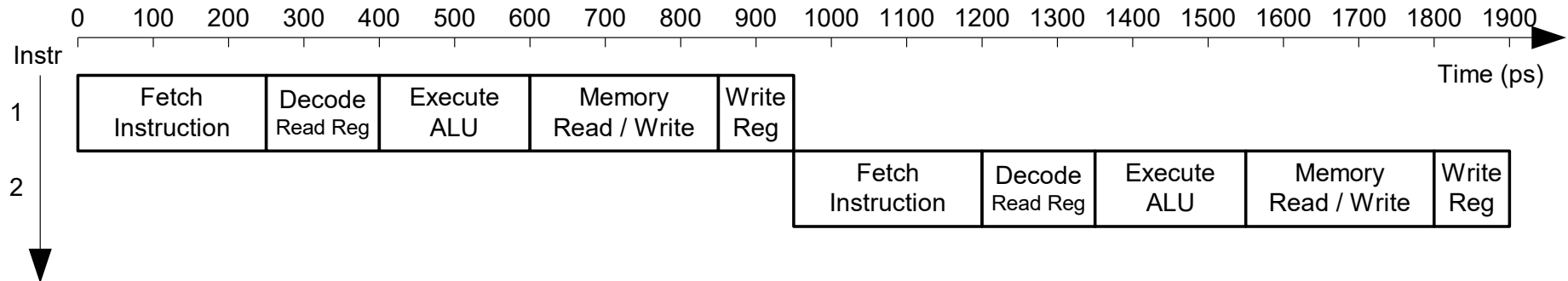


Pipelined MIPS Processor

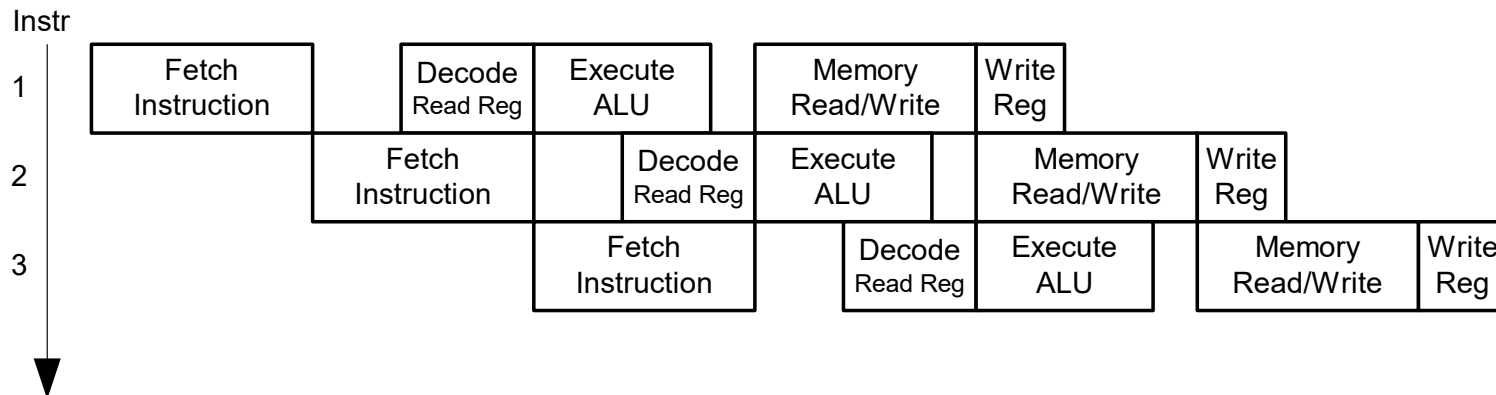
- Temporal parallelism
- Divide single-cycle processor into 5 stages:
 - Fetch
 - Decode
 - Execute
 - Memory
 - Writeback
- Add pipeline registers between stages

Single-Cycle vs. Pipelined

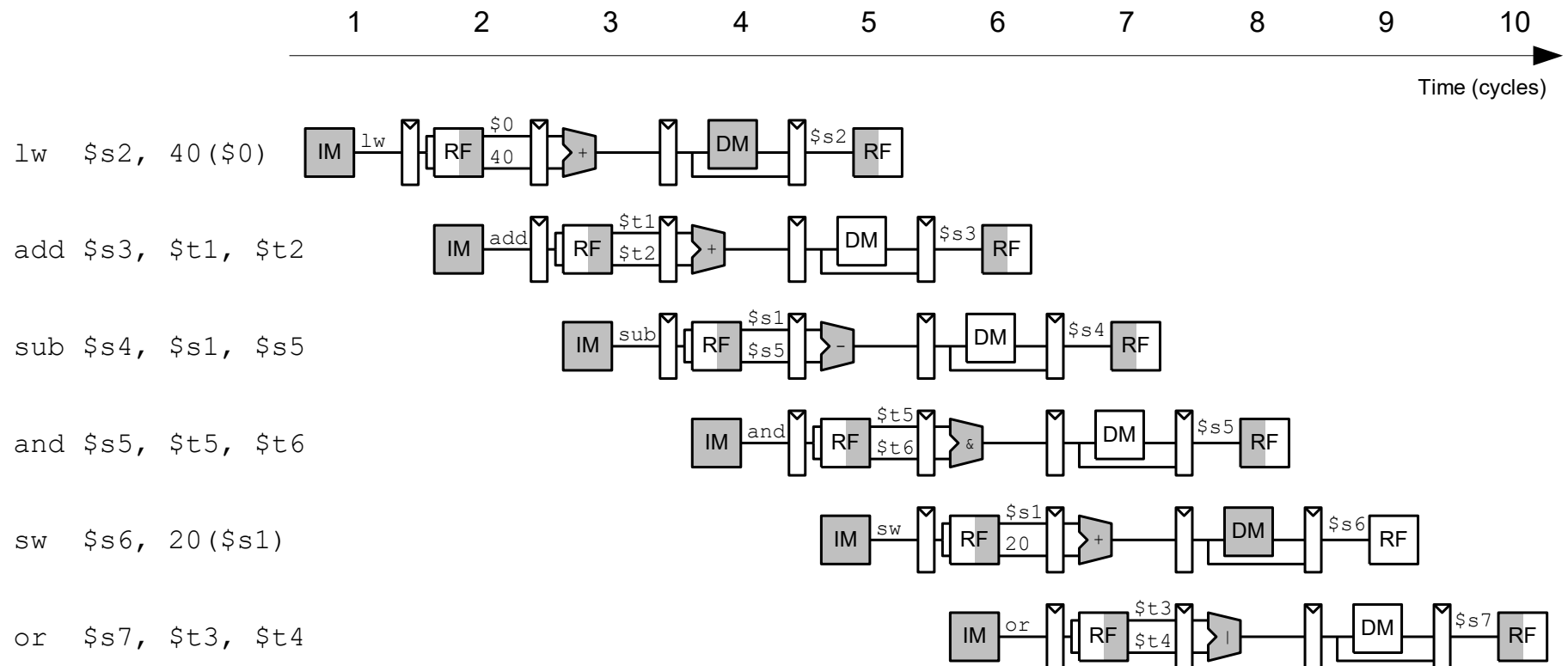
Single-Cycle



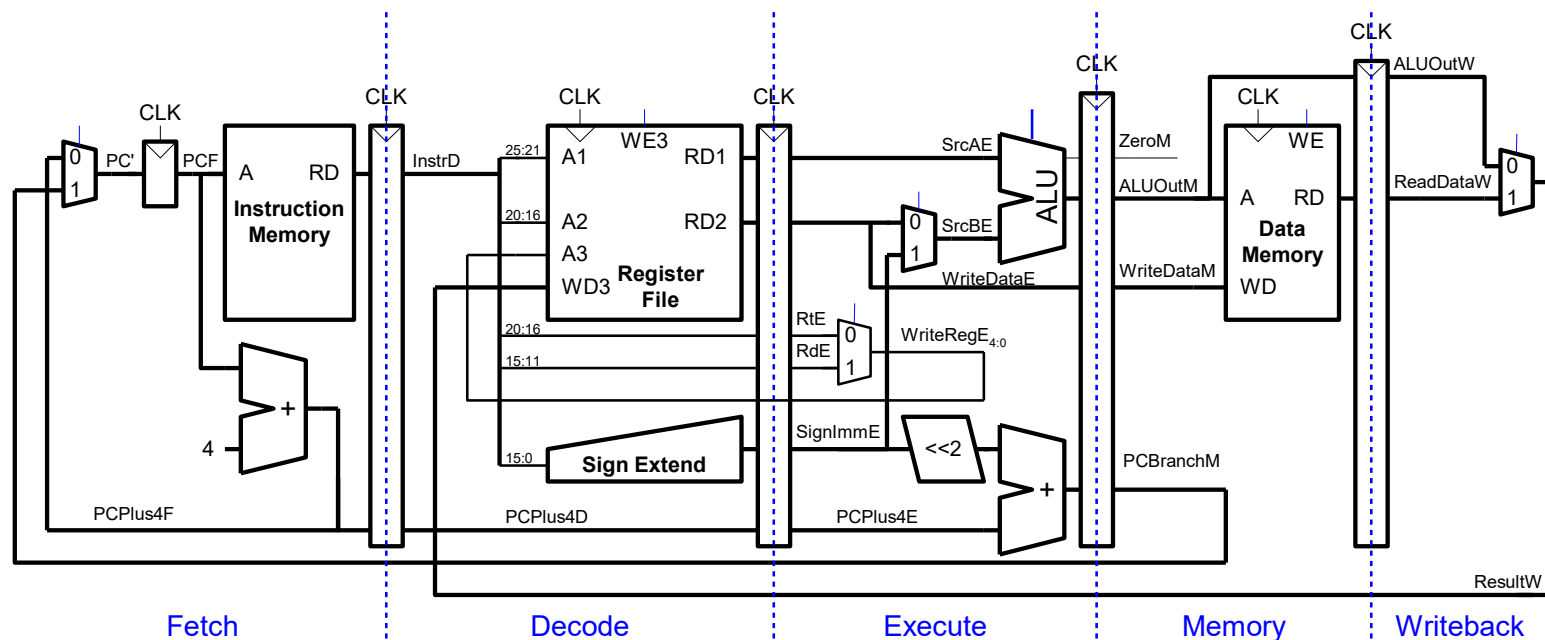
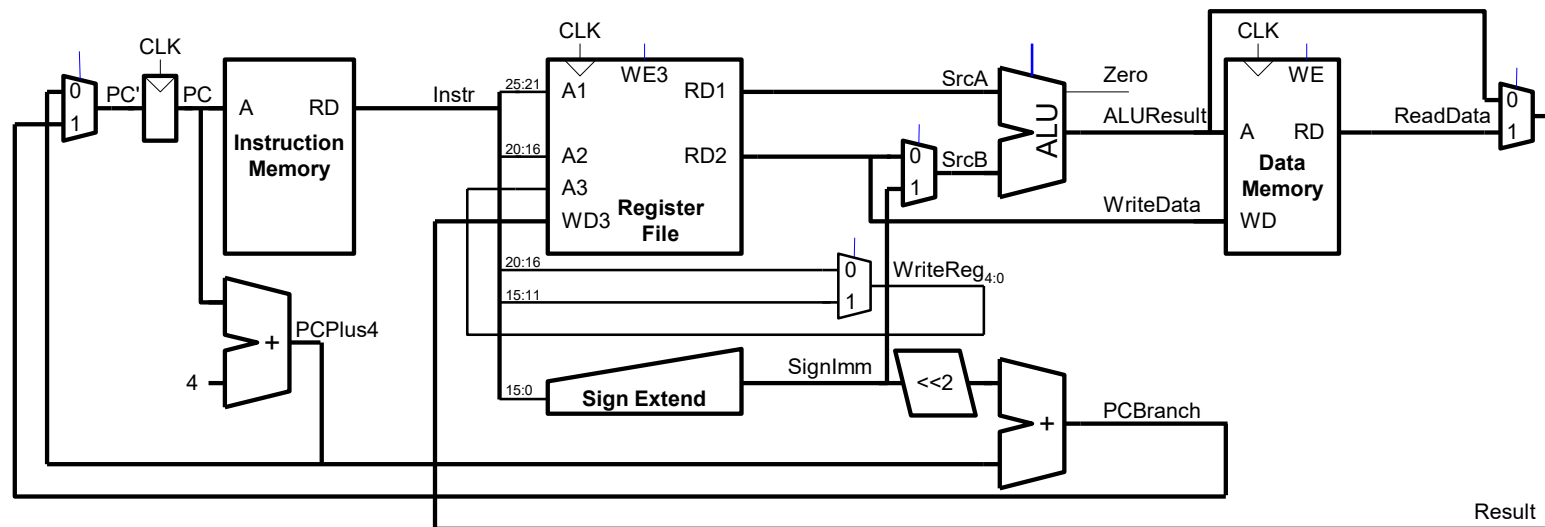
Pipelined



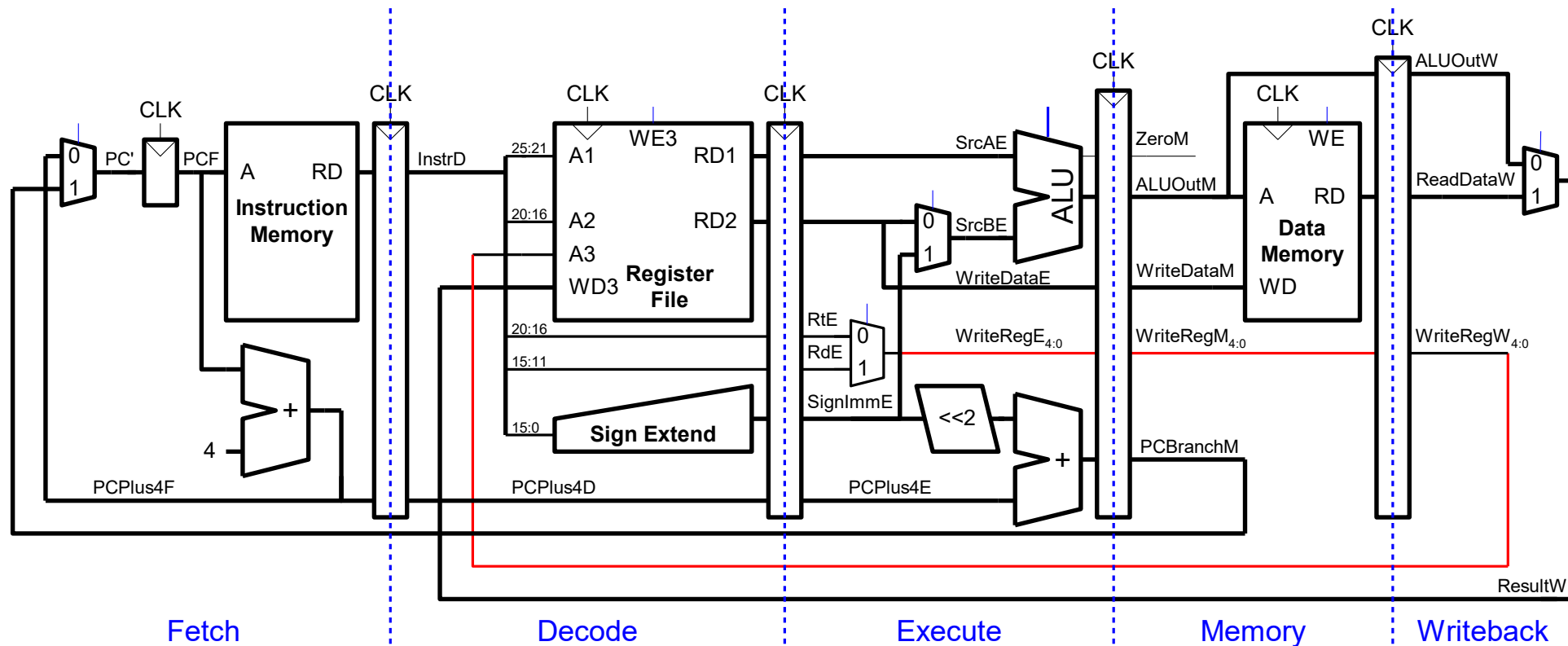
Pipelined Processor Abstraction



Single-Cycle & Pipelined Datapath

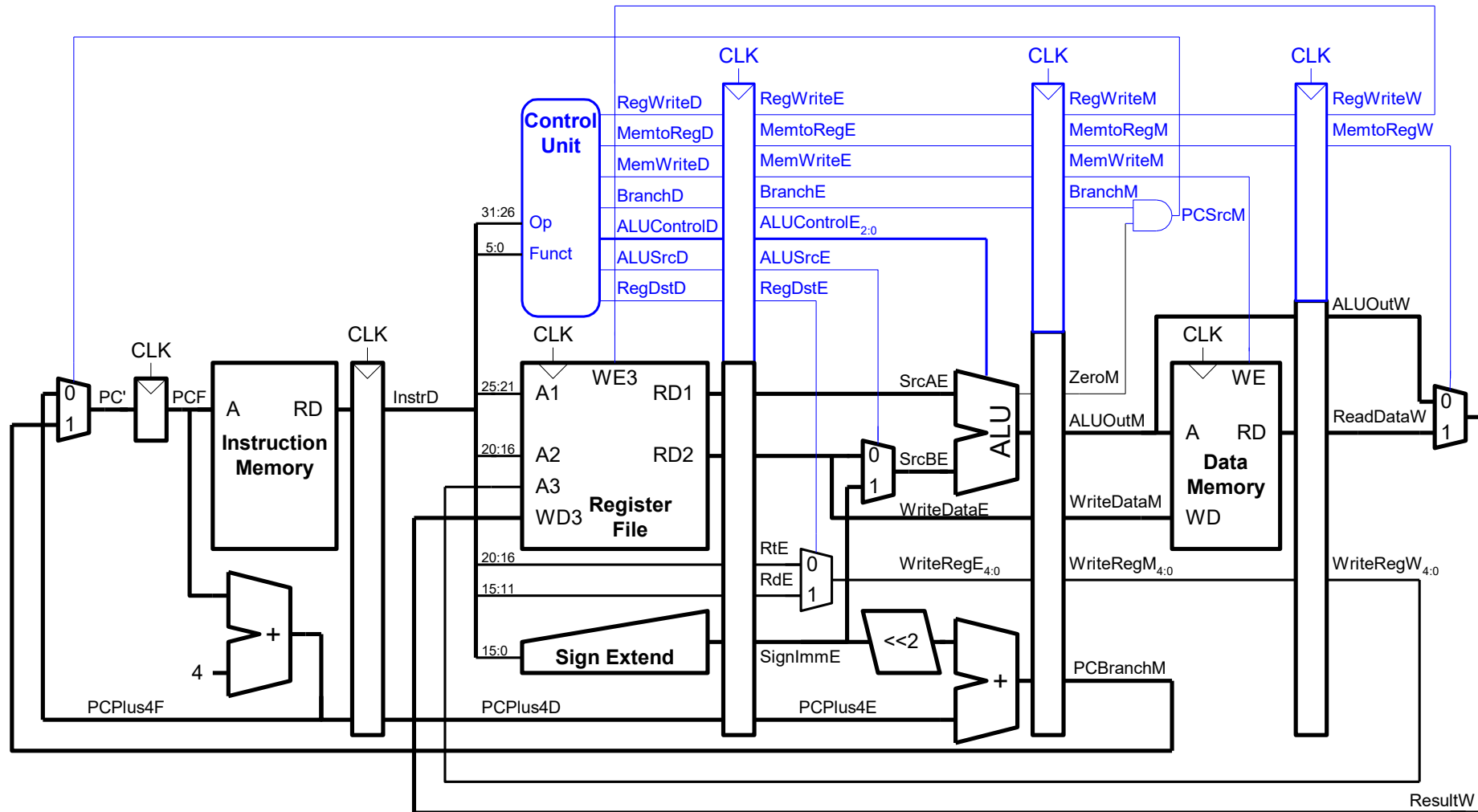


Corrected Pipelined Datapath



WriteReg* must arrive at same time as *Result

Pipelined Processor Control

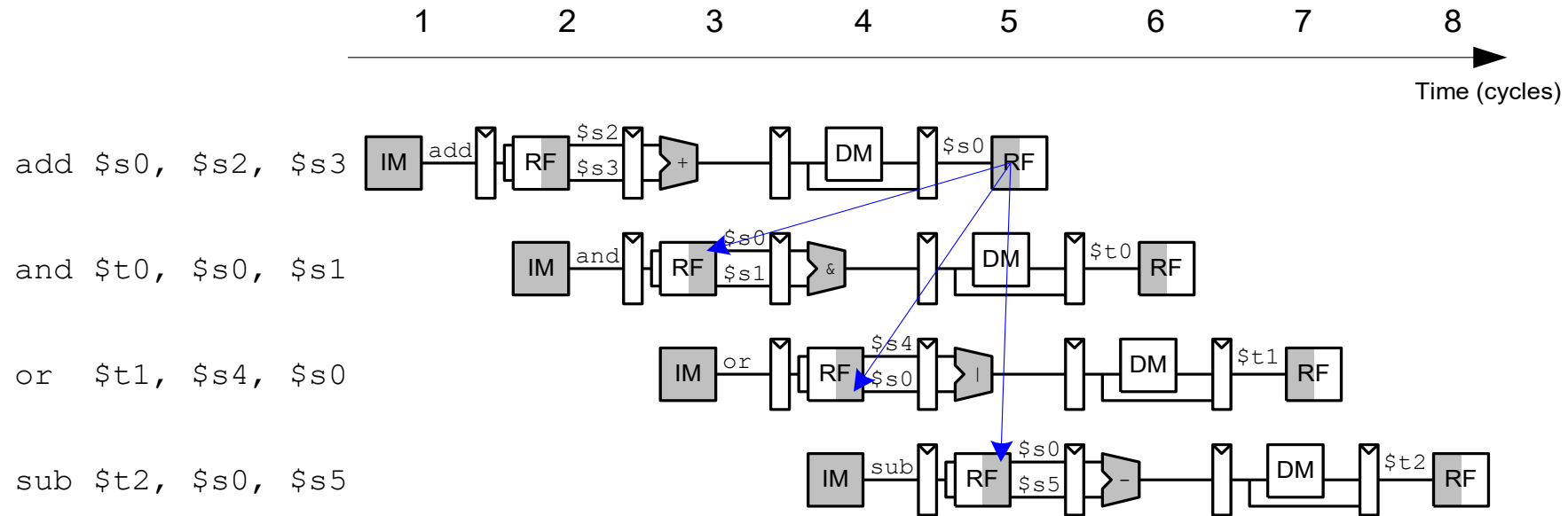


- Same control unit as single-cycle processor
- Control delayed to proper pipeline stage

Pipeline Hazards

- When an instruction depends on result from instruction that hasn't completed
- Types:
 - **Data hazard:** register value not yet written back to register file
 - **Control hazard:** next instruction not decided yet (caused by branches)

Data Hazard

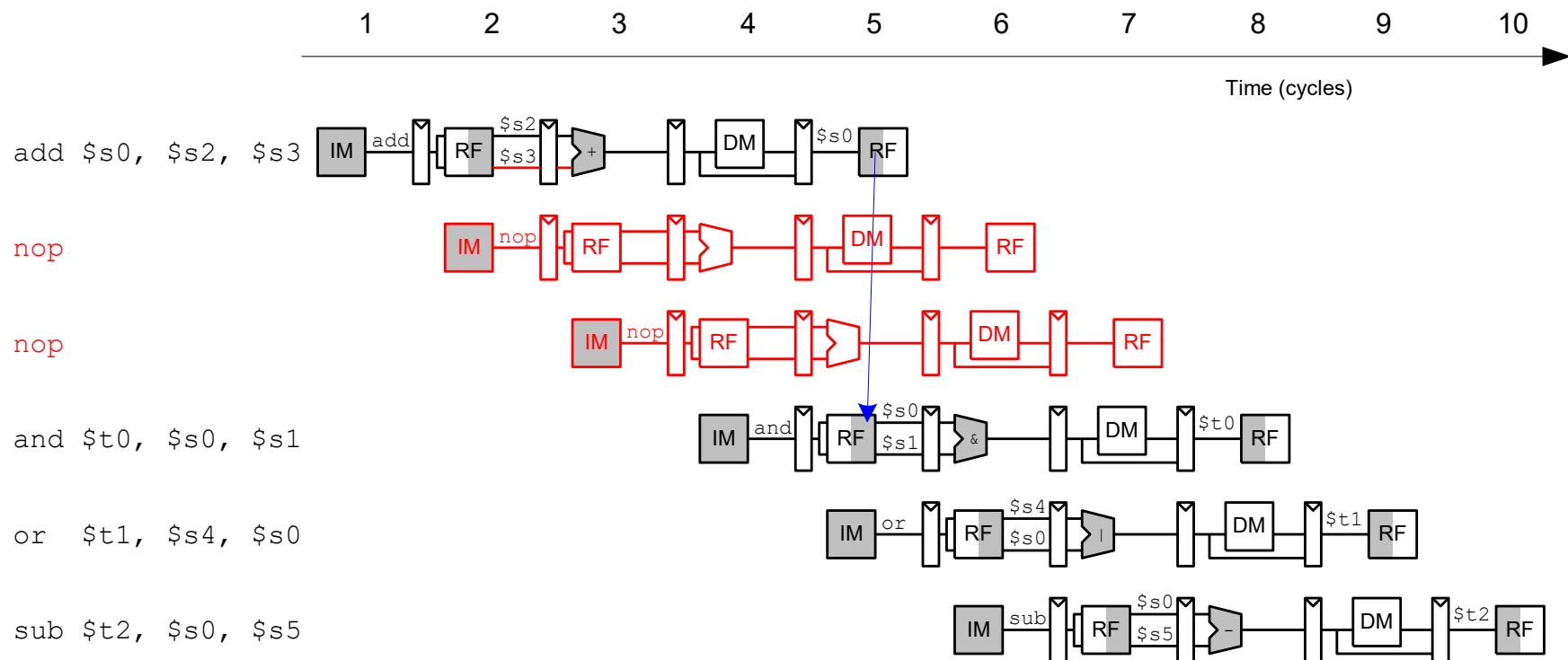


Handling Data Hazards

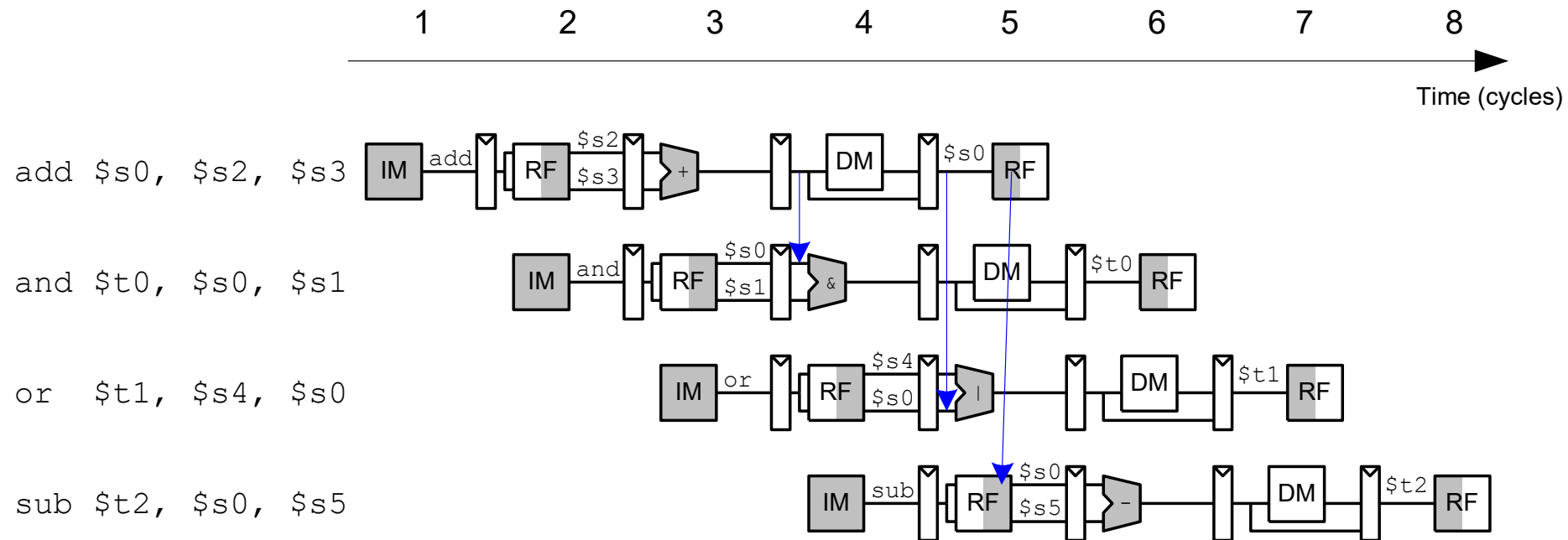
- Insert `nops` in code at compile time
- Rearrange code at compile time
- Forward data at run time
- Stall the processor at run time

Compile-Time Hazard Elimination

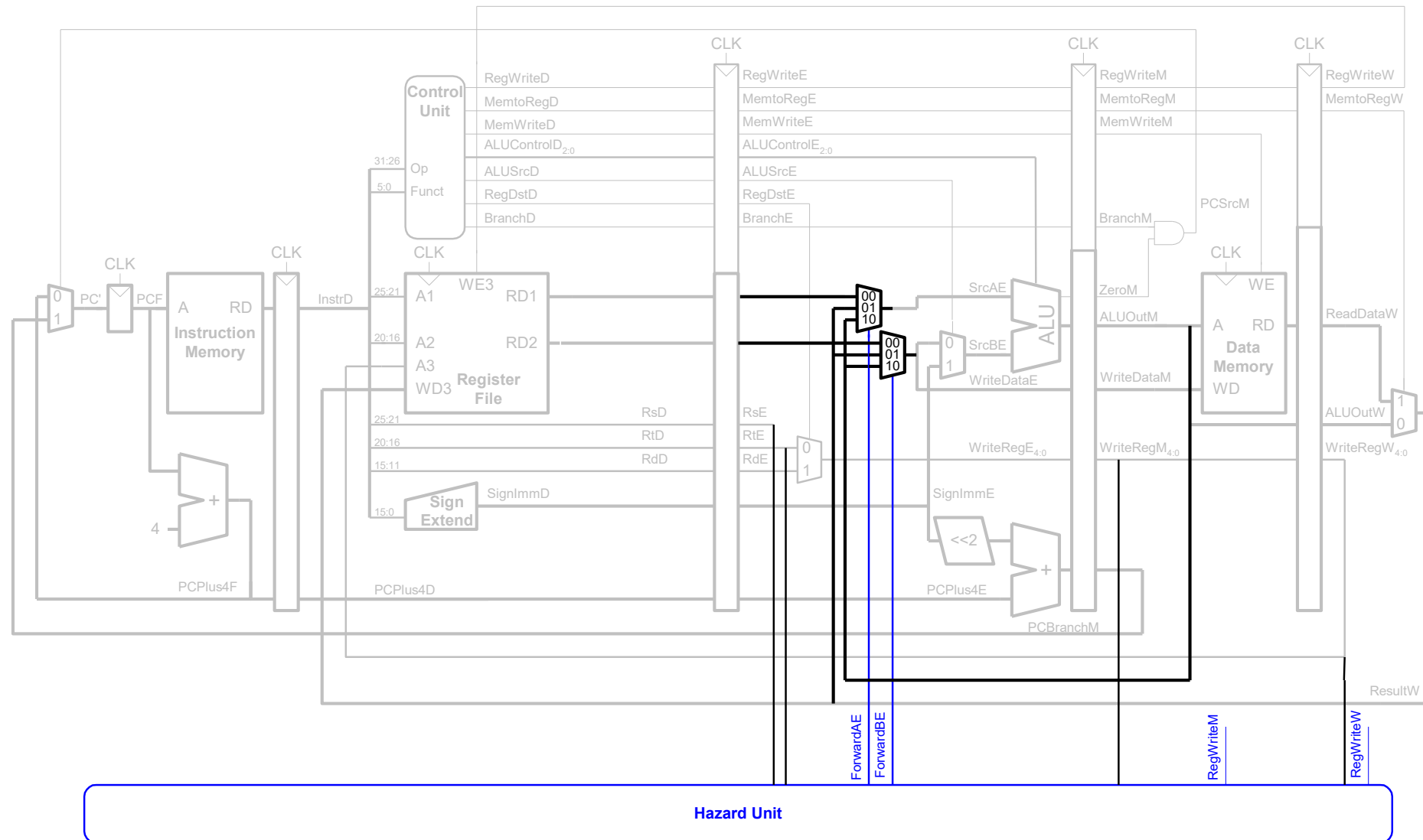
- Insert enough nops for result to be ready
- Or move independent useful instructions forward



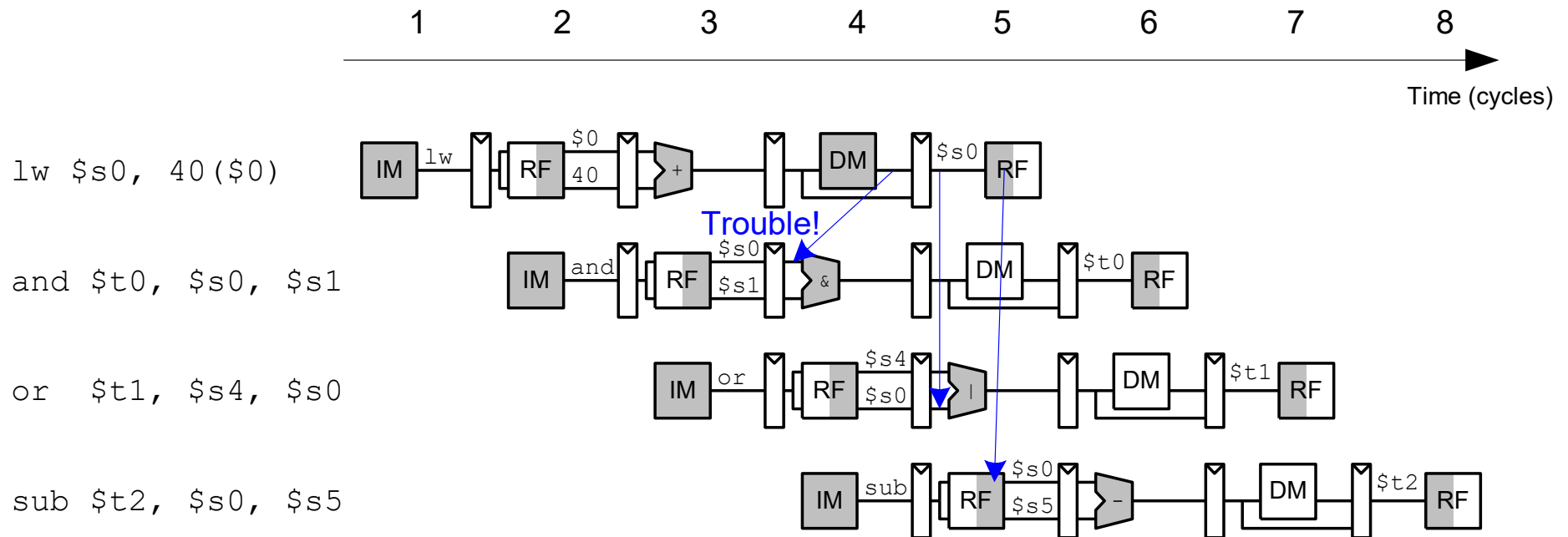
Data Forwarding



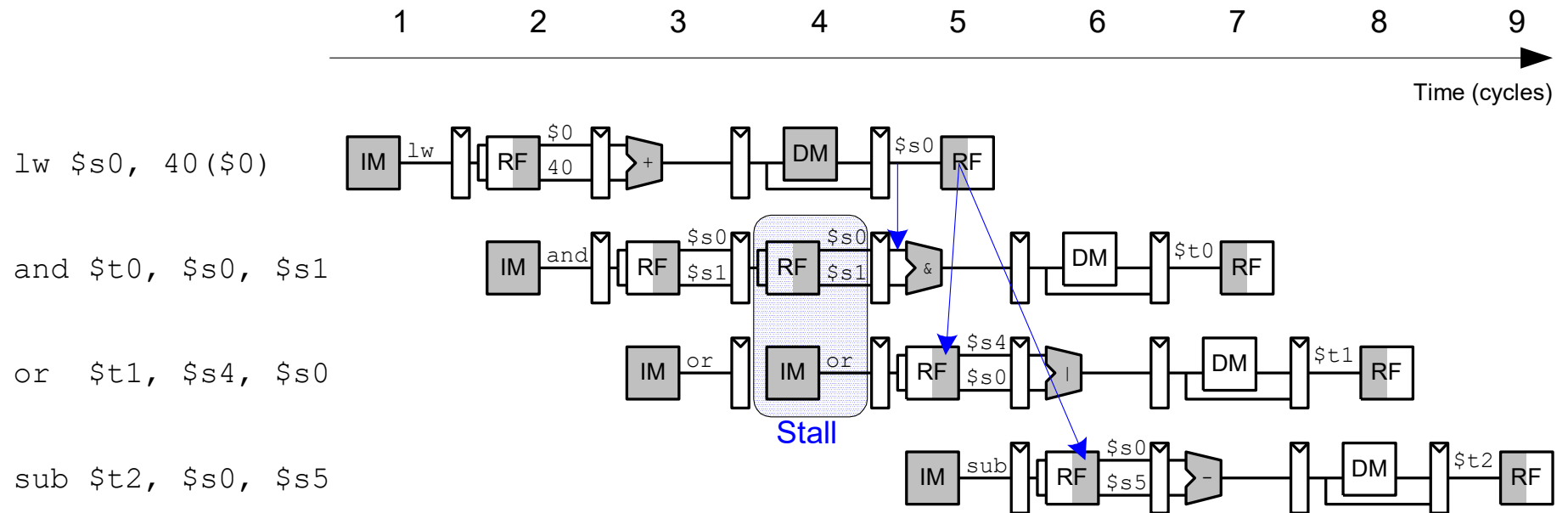
Data Forwarding



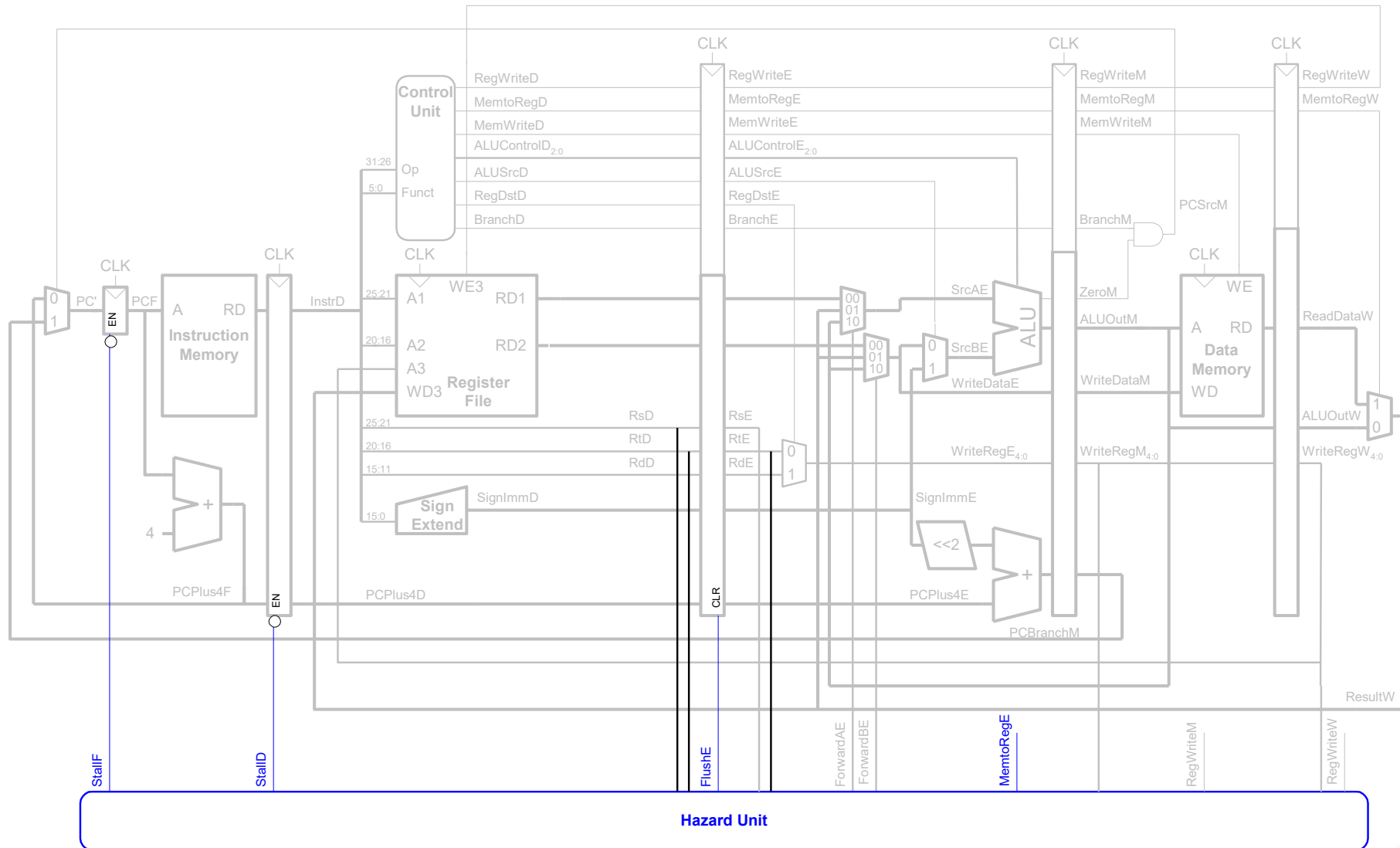
Stalling



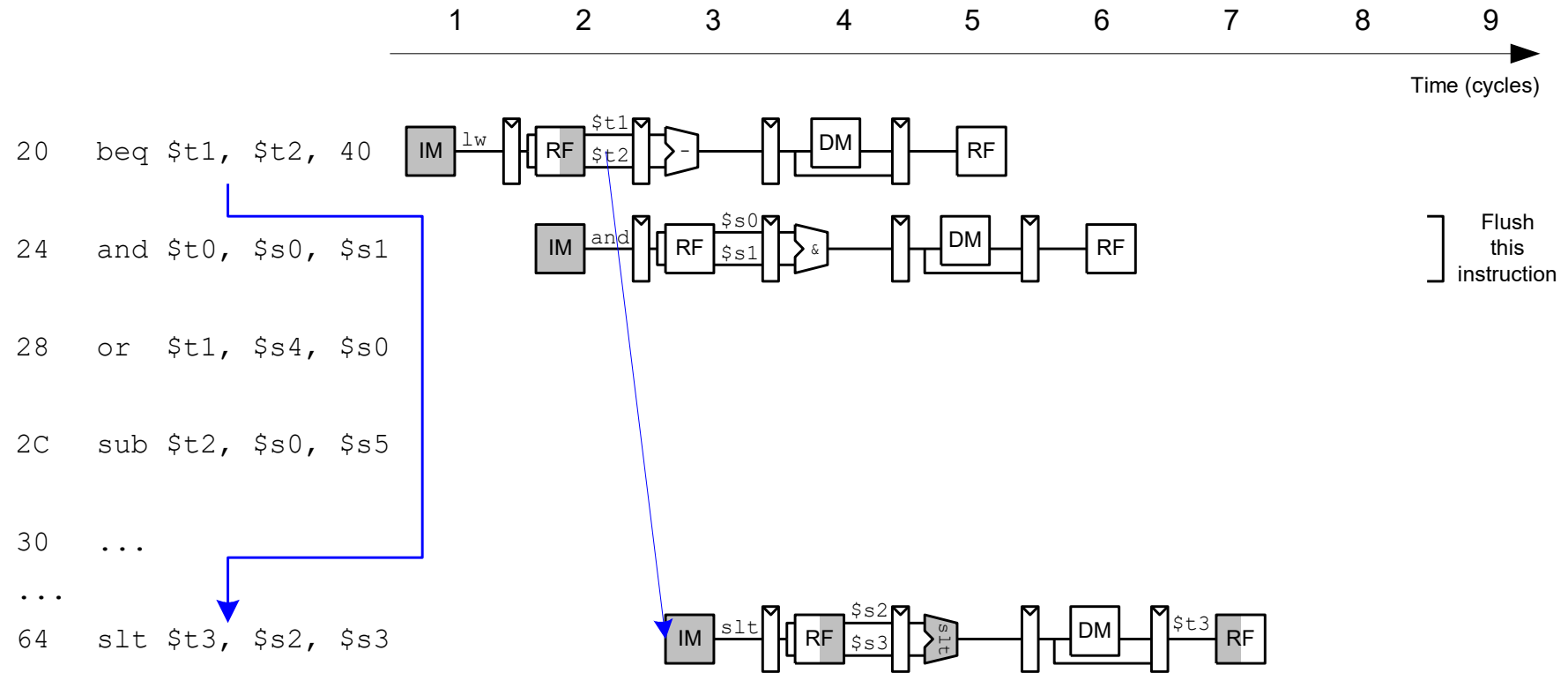
Stalling



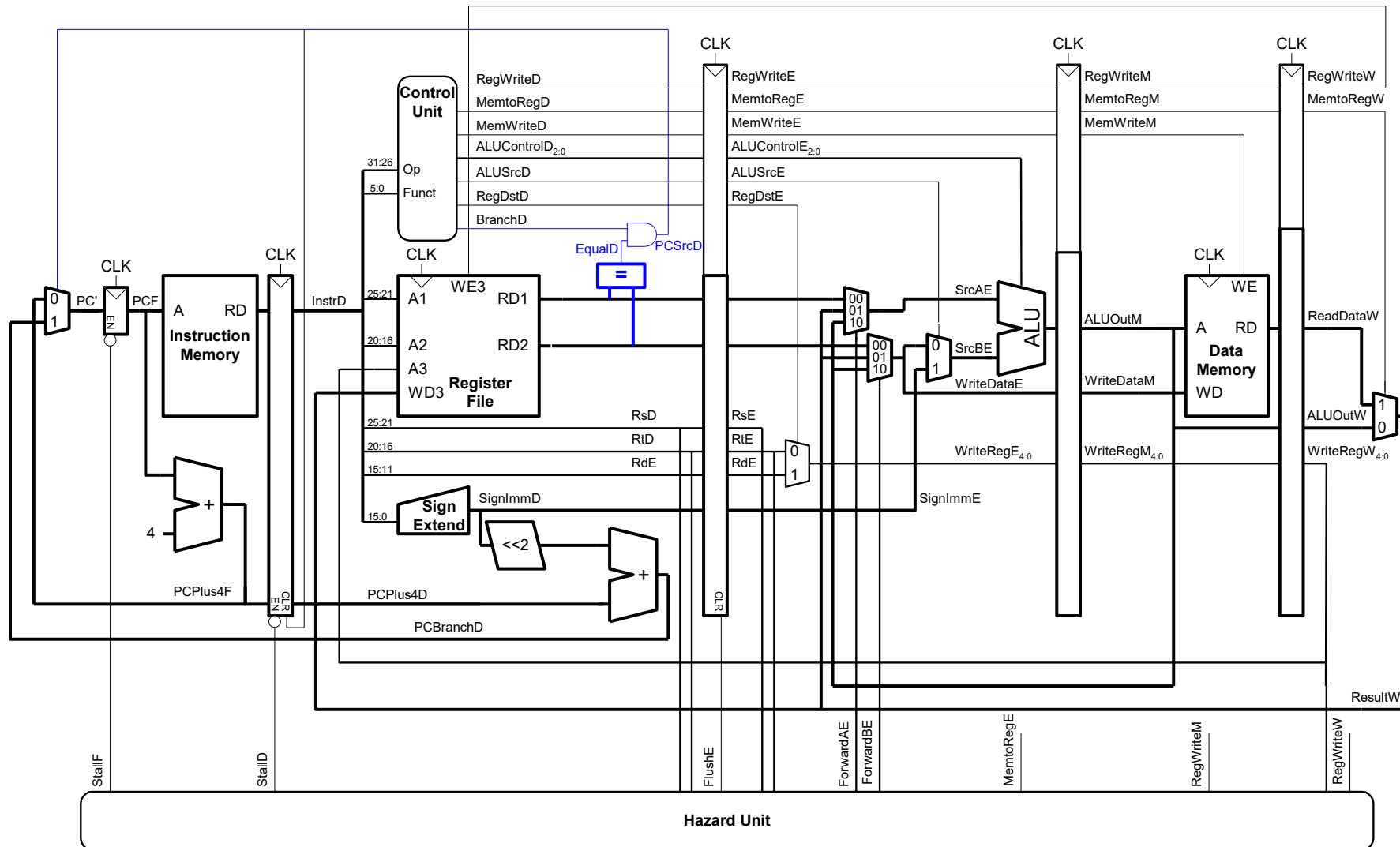
Stalling Hardware



Early Branch Resolution



Early Branch Resolution



Introduced another data hazard in Decode stage

Branch Prediction

- Guess whether branch will be taken
 - Backward branches are usually taken (loops)
 - Consider history to improve guess
- Good prediction reduces fraction of branches requiring a flush

Pipelined Performance Example

- SPECINT2000 benchmark:
 - 25% loads
 - 10% stores
 - 11% branches
 - 2% jumps
 - 52% R-type
- Suppose:
 - 40% of loads used by next instruction
 - 25% of branches mispredicted
 - All jumps flush next instruction
- **What is the average CPI?**
 - Load/Branch CPI = 1 when no stalling, 2 when stalling
 - $CPI_{lw} = 1(0.6) + 2(0.4) = 1.4$
 - $CPI_{beq} = 1(0.75) + 2(0.25) = 1.25$

$$\begin{aligned}\text{Average CPI} &= (0.25)(1.4) + (0.1)(1) + (0.11)(1.25) + (0.02)(2) + (0.52)(1) \\ &= 1.15\end{aligned}$$

Pipelined Performance

- Pipelined processor critical path:

$$T_c = \max \{$$
$$t_{pcq} + t_{mem} + t_{setup},$$
$$2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup}),$$
$$t_{pcq} + t_{mux} + t_{mux} + t_{ALU} + t_{setup},$$
$$t_{pcq} + t_{memwrite} + t_{setup},$$
$$2(t_{pcq} + t_{mux} + t_{RFwrite}) \}$$

Pipelined Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	$t_{pcq\text{ PC}}$	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	t_{RFsetup}	20
Equality comparator	t_{eq}	40
AND gate	t_{AND}	15
Memory write	t_{memwrite}	220
Register file write	t_{RFwrite}	100

$$\begin{aligned}T_c &= 2(t_{\text{RFread}} + t_{\text{mux}} + t_{\text{eq}} + t_{\text{AND}} + t_{\text{mux}} + t_{\text{setup}}) \\&= 2[150 + 25 + 40 + 15 + 25 + 20] \text{ ps} = \mathbf{550 \text{ ps}}\end{aligned}$$

Pipelined Performance Example

Program with 100 billion instructions

$$\begin{aligned}\textbf{Execution Time} &= (\# \text{ instructions}) \times \text{CPI} \times T_c \\ &= (100 \times 10^9)(1.15)(550 \times 10^{-12}) \\ &= \textbf{63 seconds}\end{aligned}$$

Processor Performance Comparison

Processor	Execution Time (seconds)	Speedup (single-cycle as baseline)
Single-cycle	92.5	1
Pipelined	63	1.47